TRABAJO PRACTICO - UNIDAD 5

Seguridad, auditoria y anonimización en PostgreSQL

Base de desarrollo: food_store_tp4
Motor: PostgreSQL 17.11
Fecha de cierre: 7 de octubre de 2026
Integrantes: Franco Storani, Guillemor Pinto, Matias Soto, Leonel Ponce de Leon
Link repo: https://github.com/fstoutn/bd2_tps/tree/main/TP%20Unidad%205

RESUMEN

Se implementó y verificó una política de roles para Food Store, se configuró la auditoría nativa de PostgreSQL y se realizaron pruebas controladas de operaciones y de un incidente simulado. También se construyó una tabla anonimizada y se comprobó que conserva los resultados agregados requeridos. Las pruebas se realizaron en la base de desarrollo indicada, no en producción.

PARTE A - ROLES Y PRINCIPIO DE MINIMO PRIVILEGIO

Se definieron roles funcionales sin inicio de sesión y dos roles de acceso: app_web y admin_datos. app_web hereda permisos de lectura y escritura; los roles funcionales separan las tareas de lectura, escritura, soporte y reportes.

La política verificada limita app_web en la tabla usuario a SELECT sobre id, rol, activo y created_at, e INSERT/UPDATE solo sobre nombre y email. No puede consultar nombre ni email mediante SELECT ni tiene SELECT a nivel de tabla. Esta limitación es por columna, no por fila.

rol_soporte puede consultar las columnas de usuario necesarias para atender casos y actualizar únicamente activo. rol_reportes puede consultar rol y created_at para sus agregaciones. rol_app_escritura tiene permisos de INSERT/UPDATE en las tablas operativas previstas. admin_datos conserva SELECT, INSERT, UPDATE y DELETE sobre las tablas de negocio y usuario_anon, sin TRUNCATE, TRIGGER ni REFERENCES.

Se retiraron los privilegios por defecto para tablas y secuencias futuras, de modo que cada objeto nuevo requiere una revisión y concesión explícita de permisos. La verificación final confirmó que \ddp no muestra grants automáticos y que los privilegios efectivos de app_web sobre usuario coinciden con la política definida.

Evidencia: `sql/roles.sql`; `docs/kiro_roles_spec/requirements.md`; `docs/kiro_roles_spec/design.md`; `docs/verificacion_roles_parte_a_completa.txt`.

PARTE B - AUDITORIA DE POSTGRESQL

Se verificaron y dejaron activos `log_connections` y `log_disconnections`. `log_statement` se configuró temporalmente en `all` para registrar tanto lecturas como modificaciones, y luego se restauró a `mod`. El prefijo del log incluye fecha y hora, PID, usuario, base de datos y origen de conexión.

La prueba de lectura, inserción y actualización se ejecutó dentro de una transacción finalizada con ROLLBACK. También se comprobó un intento fallido de autenticación. El log registró además una conexión autenticada por SCRAM-SHA-256 y su desconexión normal.

La auditoría nativa permite relacionar sentencias y sesiones con hora, rol, base y origen. No aporta por sí sola el contexto de negocio ni una descripción fiable de los valores anteriores y nuevos de cada fila; una sentencia registrada tampoco demuestra por sí sola que una transacción haya confirmado cambios.

Los logs pueden contener información sensible. Su acceso debe limitarse a DBA y personal autorizado de seguridad u operaciones, mediante controles del sistema operativo y del motor. Los fragmentos incluidos en la entrega están anonimizados.

Evidencia: `informe_parte_b.md`.

PARTE C - SIMULACRO Y ANALISIS DEL INCIDENTE

Como los trabajos previos no incluían fn_autenticar ni una columna de contraseña en usuario, se preparó una reconstrucción aislada para laboratorio. Usa una identidad sintética, almacena la contraseña como hash mediante pgcrypto y no modifica la tabla real de usuarios. No se afirma que reproduzca exactamente la función vista en clase.

En el simulacro, dos autenticaciones sintéticas fallaron y una tuvo éxito. Luego, app_web ejecutó una lectura ordenada de usuario que, según la salida de psql, devolvió 20.005 filas en 0,003 segundos. Finalmente, el intento de otorgar admin_datos a app_web fue rechazado por PostgreSQL. La consulta posterior de membresías devolvió cero filas para app_web como miembro de admin_datos.

La respuesta de OpenCode, el contraste humano y la decisión de contención están documentados por separado. El log compartido no identifica a una persona ni revela el origen real ni el contenido de las filas. El conteo y el tiempo provienen de la salida de psql, no del log. No hay evidencia en este material de que los datos se hayan exportado o utilizado fuera de la base.

La contención decidida para el laboratorio fue no bloquear cuentas ni aplicar acciones sobre producción: la elevación de privilegios falló y la prueba usó una identidad sintética. En un caso real se preservarían los logs, se investigaría el alcance del acceso y se tomarían medidas adicionales solo de acuerdo con la evidencia confirmada.

Evidencia: `sql/reconstruir_autenticacion_lab.sql`; `docs/log_simulacro_anonimizado.txt`; `docs/respuesta_opencode_parte_c.md`; `docs/analisis_incidente_parte_c.md`.

PARTE D - ANONIMIZACION Y EQUIVALENCIA

Se creó usuario_anon con identificadores sintéticos deterministas, conservando rol y mes de alta para las consultas agregadas. No se copiaron los identificadores, nombres ni correos reales. El script se detiene si la tabla ya existe para evitar reemplazar datos sin revisión.

La tabla anonimizada contiene 20.005 filas. Los agregados por rol y mes coinciden con los de usuario:

Rol	Mes de alta	Usuario	Usuario anonimizado
CLIENTE	2026-09-01 00:00:00-03	20.004	20.004
ADMIN	2026-09-01 00:00:00-03	1	1

La consulta final de diferencias devolvió cero filas. No deben compartirse con asistentes de IA nombres, correos, direcciones, identificadores personales, credenciales, datos sensibles de clientes o pedidos, ni combinaciones que permitan reidentificar a una persona. La evidencia debe anonimizarse antes de salir del entorno controlado.

Evidencia: `sql/usuario_anon.sql`; `sql/verificar_equivalencia_usuario_anon.sql`; `docs/verificacion_equivalencia.md`.

PARTE E - BITACORA Y CONTROL DE VERSIONES

BITACORA.md reúne los prompts, respuestas relevantes y validaciones humanas de A-D. También describe la recomendación de privilegios por defecto demasiado amplios, cómo se detectó al revisar \ddp y la corrección aplicada.

El historial contiene commits separados para las partes:

```
099295d  Se realiza Parte A: roles y permisos
f1a54d7  Se realiza Parte B: auditoria de PostgreSQL
0f45264  Se realiza Parte C: simulacro y analisis de incidente
e0027fb  Se realiza Parte D: anonimización y equivalencia
d7c88ac  Se realiza Parte E: bitacora y cierre del TP
```

Posteriormente se registró `6b5b228` para eliminar un archivo incorrecto. En la última verificación informada, main y origin/main estaban sincronizadas y git status reportó el árbol de trabajo limpio. Esta reformulación del informe es posterior a esa verificación y queda pendiente de publicar.

CONCLUSION

Las partes A-D quedaron implementadas y verificadas en el entorno de desarrollo. La política de roles aplica permisos acotados; la auditoría registra los eventos requeridos con límites conocidos; el simulacro confirmó que app_web no pudo escalar a admin_datos; y la tabla anonimizada preservó los agregados solicitados sin conservar identificadores personales directos.